

SISTEMI CLOUD · PROGETTO ABSA

absa-cloud

Architettura e deploy su AWS

Piattaforma 3-tier a microservizi per l'Aspect-Based Sentiment Analysis di recensioni bancarie, portata dall'ambiente locale su Kubernetes ad Amazon EKS.



Il problema / inferenza lenta su CPU

INFERENZA REALE

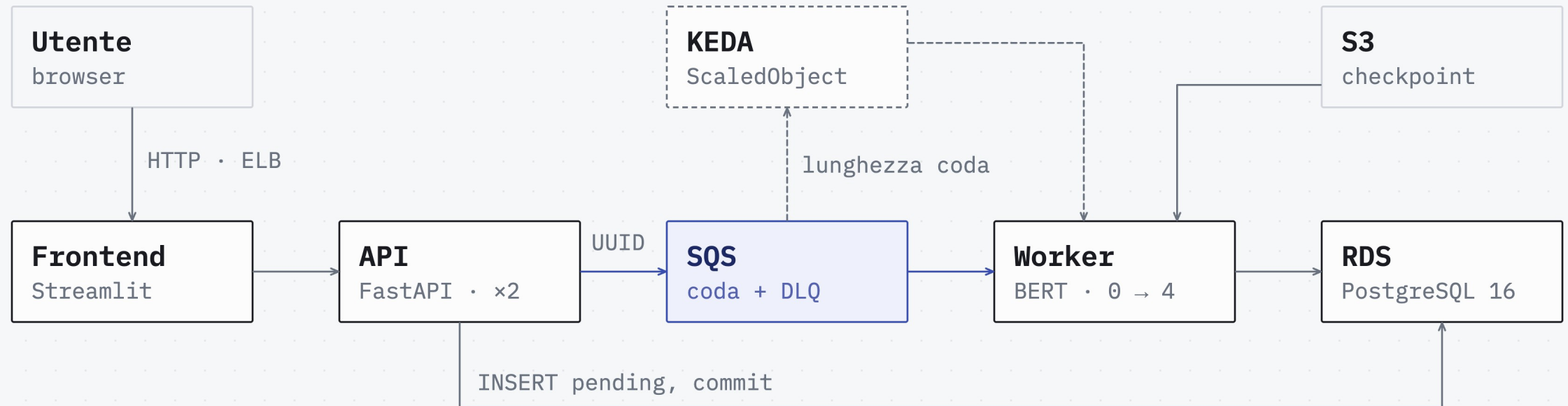
~4 s

per recensione, modello BERT
su CPU

LCF-ATEPC • checkpoint 450 MB

- Un'API sincrona terrebbe il client in attesa per secondi, e sotto un picco non reggerebbe.
- L'API salva la recensione come **pending**, pubblica in coda il solo UUID e risponde subito **202 Accepted**.
- I worker consumano al proprio ritmo: la **lunghezza della coda** è il segnale che li fa scalare.
- Scalare i worker vuol dire scalare l'inferenza, senza toccare API e database.

Architettura / il giro di una recensione



FASE A In locale la topologia è la stessa su OrbStack: RabbitMQ al posto di SQS, Postgres in un pod con volume, modello da una cartella montata.

Da OrbStack ad AWS / stesso codice

Componente	Fase A • locale	Fase B • AWS
Cluster	OrbStack, Kubernetes sul Mac	Amazon EKS 1.34, nodi spot
Coda	RabbitMQ in un pod	SQS con dead letter queue
Database	Postgres in un pod con PVC	RDS PostgreSQL 16, subnet privata
Immagini	build locale, nessun registry	ECR, pubblicate dalla CI
Modello	cartella del Mac montata nel pod	S3, scaricato all'avvio del worker
Credenziali	in chiaro nei manifest	IRSA: ruoli IAM, nessuna chiave
Esposizione	LoadBalancer di OrbStack	load balancer AWS in subnet pubbliche
Codice applicativo	identico	identico: cambiano due variabili d'ambiente

Due stack Terraform / persistent e compute

terraform/aws/persistent

persistent si crea una volta

S3	bucket del modello, versioning, accesso pubblico bloccato
ECR	3 repository, scan on push, ultime 10 immagini
IAM OIDC	ruolo della CI per il solo push su ECR
Budgets	avvisi via email a 10, 50 e 100 \$

costo: pochi centesimi al mese

terraform/aws/compute

compute nasce e muore con la sessione

VPC	2 zone di disponibilità, un NAT gateway
EKS	Kubernetes 1.34, node group spot
RDS	PostgreSQL 16 in subnet privata
SQS	coda principale e dead letter queue
IAM IRSA	3 ruoli: api, worker, KEDA
KEDA	installato con Helm

costo: ~0,43 \$ l'ora

Terraform → Ansible → K8s / chi passa cosa

01 · CREA

Terraform

crea e dichiara

sqs_queue_url
database_url
api_role_arn
worker_role_arn
model_bucket
ecr_repository_urls



02 · TRASPORTA

Ansible

legge e inietta

Il modulo **terraform_output** legge gli output di entrambi gli stack e li raccoglie nelle variabili di deploy.

Nessun valore copiato a mano in un file.



03 · CONSUMA

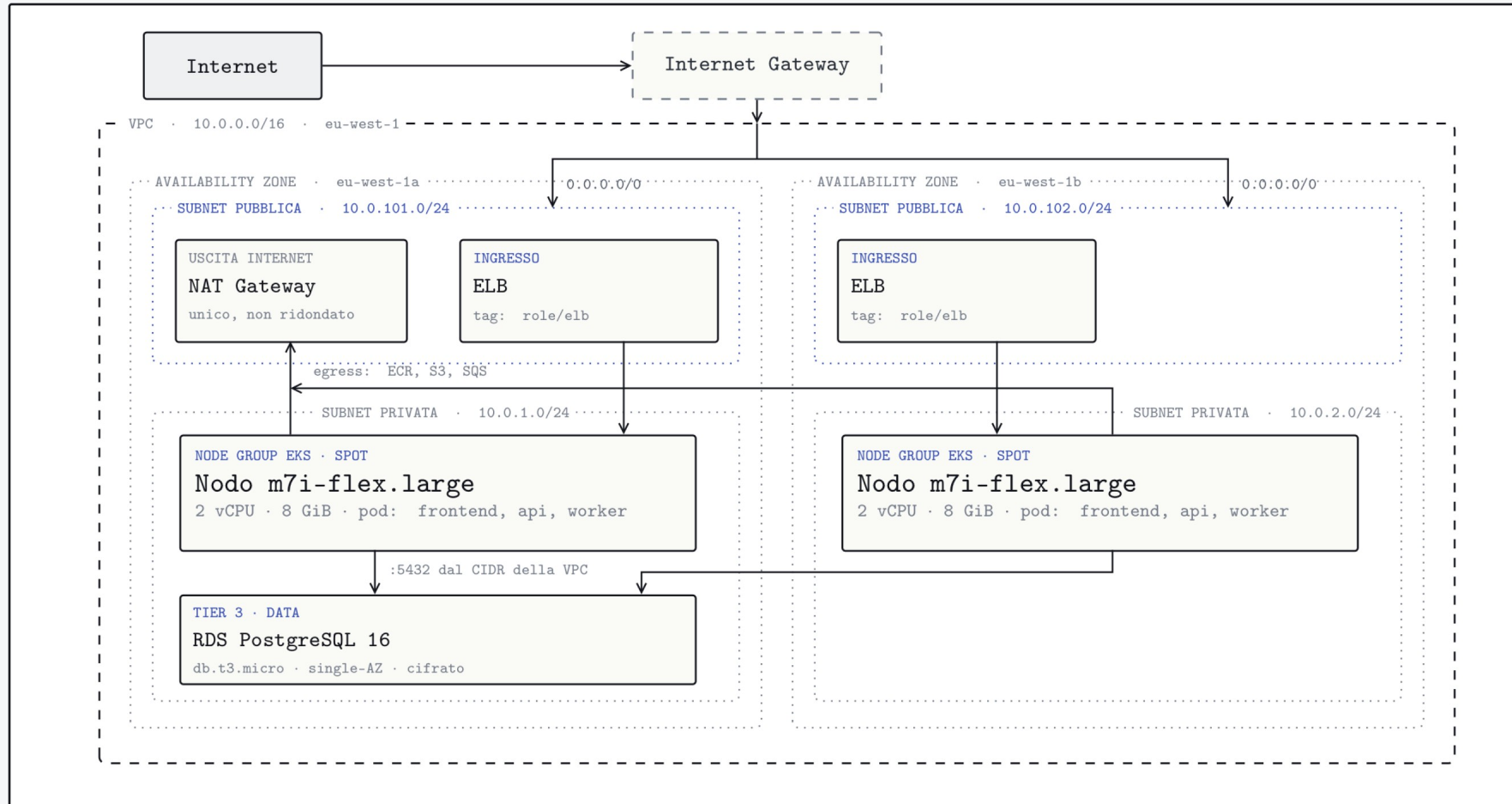
Kubernetes

manifest con segnaposto

`${SQS_QUEUE_URL}`
`${WORKER_ROLE_ARN}`
`${IMAGE_WORKER}`
`${DATABASE_URL}`

render_manifest.py sostituisce solo `${VAR}`, lascia `$VAR` ai container e si ferma se un valore manca.

Rete / VPC su due zone



Cluster EKS / nodi spot

- Modulo ufficiale **terraform-aws-modules/eks**, con un node group gestito di istanze spot.
- Dimensionato sul worker con il modello reale: BERT e PyTorch occupano circa **2 GiB per pod**.
- Il limite per nodo è la **CPU**: ogni worker chiede 500m e, con i pod di sistema, su 2 vCPU ne entrano due.

NODE GROUP GESTITO

EKS

absa-cluster

VERSIONE	1.34
ISTANZA	m7i-flex.large
RISORSE	2 vCPU · 8 GiB
CAPACITÀ	SPOT
NODI	3

Nodi spot / perché reggono

Il lavoro non vive nel pod: resta nella coda finché il worker non lo conferma.

01 · INTERRUZIONE

AWS può riprendersi un'istanza spot con due minuti di preavviso.

02 · RICONSEGNA

I messaggi non ancora confermati tornano in coda e li prende un altro worker.

03 · RIPRISTINO

Il pod perso viene ricreato e il node group rimpiazza il nodo per tornare a tre.

RDS PostgreSQL / il tier dati

- In **subnet private**, mai raggiungibile da internet: il security group apre la 5432 solo al CIDR della VPC.
- Dal portatile quindi non si arriva: lo schema lo applica un **Job dentro il cluster**, con kubectl wait come barriera prima di API e worker.
- Single-AZ e nessuno snapshot finale: rinunce dichiarate per un database che a fine sessione è usa e getta.

TIER DATI

RDS

subnet group su 2 zone

MOTORE	PostgreSQL 16
CLASSE	db.t3.micro
STORAGE	20 GB cifrati
ZONE	single-AZ
ACCESSO	5432 · solo VPC

SQS / coda e dead letter queue

- **Visibility timeout 300 s**, il tempo massimo di un batch con il modello vero: più corto, doppie elaborazioni; più lungo, messaggi fermi su un worker morto.
- **Dead letter queue dopo 5 consegne**: un messaggio che fa cadere il worker viene messo da parte invece di girare per sempre.
- Long polling di 20 s e batch da 10: meno chiamate, che su SQS si pagano.

CODA · SQS

absa-reviews

+ absa-reviews-dlq

VISIBILITY 300 s

RETENTION 4 giorni

DLQ 5 consegne · 14 gg

TIPO standard

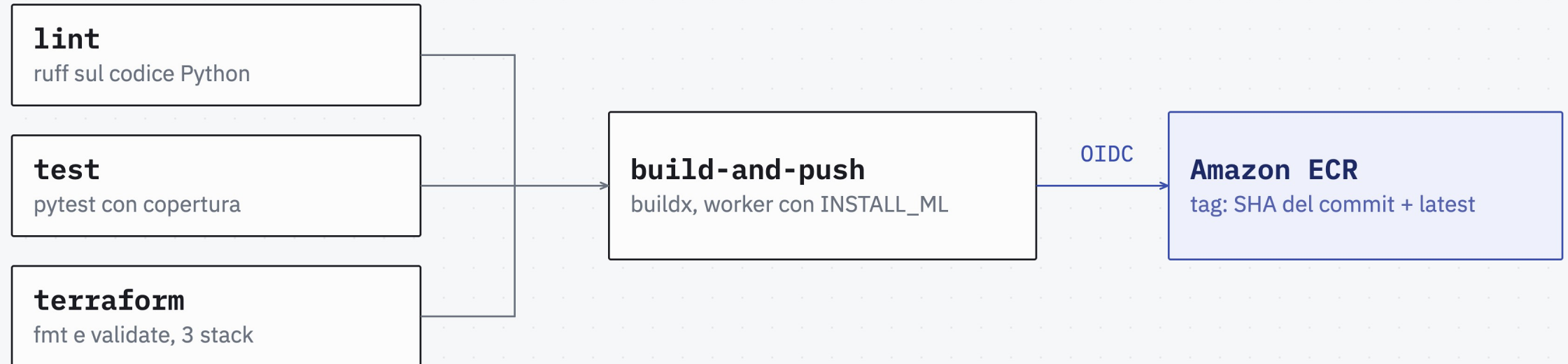
POLLING 20 s · batch 10

IRSA / un ruolo in base allo scopo

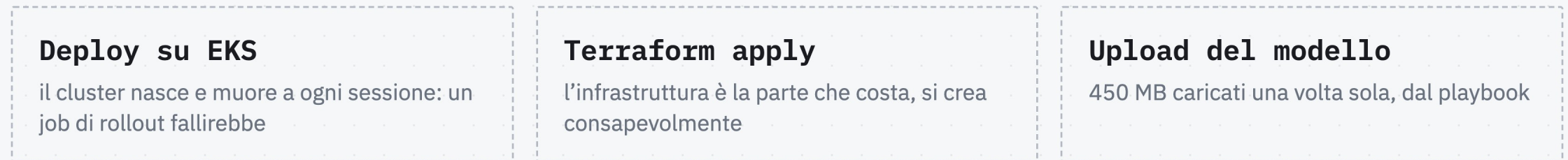
ServiceAccount	Ruolo IAM	Può	Non può
absa-api	absa-api-irsa	inviare messaggi, leggere gli attributi della coda	consumare dalla coda
absa-worker	absa-worker-irsa	ricevere e cancellare messaggi, leggere il bucket del modello	pubblicare nuovi messaggi
keda-operator (namespace keda)	absa-keda-irsa	leggere gli attributi della coda	toccare i messaggi
frontend	nessuno	—	nessuna chiamata ad AWS

Trust policy di ogni ruolo: `sub = system:serviceaccount:absa-cloud:absa-worker`

Pipeline CI/CD / GitHub Actions



VOLUTAMENTE FUORI DALLA PIPELINE

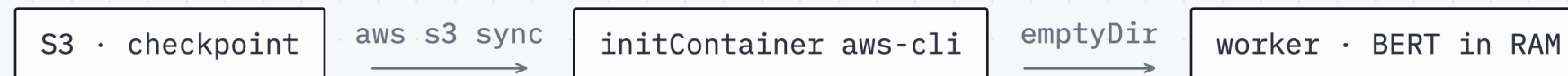


Deploy con Ansible / playbook AWS

- 01** Legge gli output dei due stack con **terraform_output**: coda, ruoli, database, bucket, registry.
- 02** Aggiorna il kubeconfig del cluster EKS.
- 03** Build e push delle tre immagini con **--platform linux/amd64**: il Mac è ARM, i nodi x86.
- 04** Carica il checkpoint su S3, solo con **upload_model=true**.
- 05** Renderizza e applica namespace, config e ServiceAccount con gli ARN IRSA.
- 06** Job **db-init** su RDS e **kubectl wait**: barriera prima dell'applicazione.
- 07** Applica API, worker e ScaledObject con gli URI delle immagini su ECR.
- 08** Attende i rollout e stampa gli indirizzi dei due load balancer.

— Con **-e build_images=false** salta la build e usa le immagini già pubblicate dalla CI.

Worker su EKS / modello da S3



- Il modello resta fuori dall'immagine: il codice cambia spesso, i **450 MB** di pesi quasi mai.
- Nessuna chiave nel pod: la lettura del bucket passa dal **ruolo IRSA** del worker.
- Il prezzo è l'avvio: il pod consuma la coda solo dopo aver scaricato il modello e caricato BERT in RAM.

DEPLOYMENT WORKER

REQUEST	500m · 2 GiB
LIMIT	2 CPU · 4 GiB
BATCH	10 messaggi
GRAZIA	120 s
ACK	DeleteMessage

KEDA su SQS / cooldown e cold start

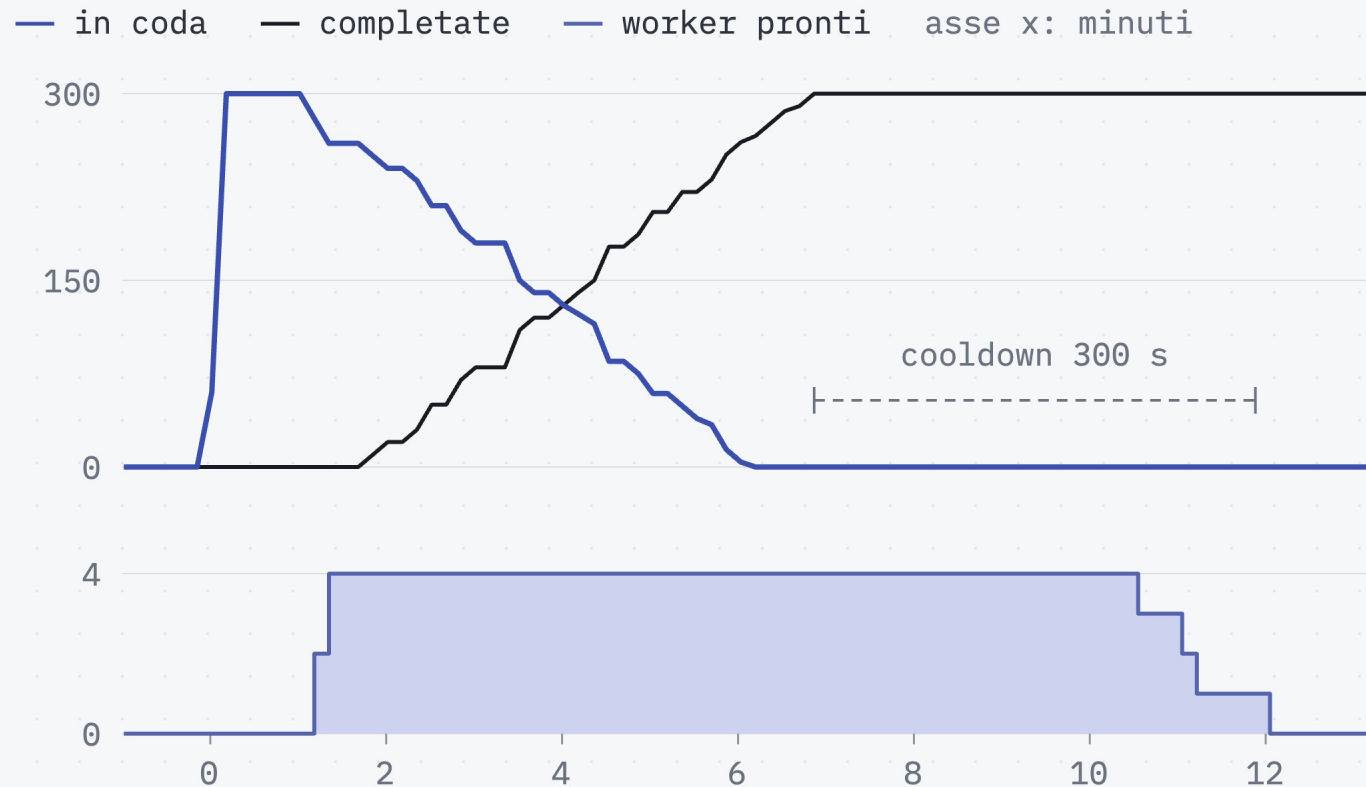


- **Cooldown 300 s:** un worker impiega circa 45 s ad accendersi (immagine, modello da S3, BERT in RAM). Con un cooldown breve ogni recensione isolata pagherebbe il cold start.
- **Tetto a 4:** ogni worker chiede mezza CPU e i nodi ne hanno due.

TRIGGER · AWS-SQS-QUEUE

VALUE	30 msg / pod
REPLICHE	0 → 4 · locale 5
POLLING	15 s · locale 10
COOLDOWN	300 s · locale 30
IDENTITÀ	operator

Risultati / burst da 300 recensioni



burst 300 • concorrenza 30 • pyabsa

300 / 300

risposte 202 Accepted

58 req/s

throughput di ingestione

~35 s

worker da richiesti a pronti

372 s

svuotamento della coda

301 s

rientro a zero repliche

Costi / stack compute

Risorsa (stack compute)	Costo orario
Control plane EKS	~0,10 \$
3 × m7i-flex.large spot	~0,18 \$
NAT gateway	~0,05 \$
RDS db.t3.micro	~0,02 \$
2 × load balancer	~0,06 \$
IPv4 pubblici	~0,02 \$
SQS, S3, ECR	~0 \$
Totale	~0,43 \$

SESSIONE DI 4 ORE

~1,7 \$

COMPUTE DIMENTICATO ACCESO

~10 \$ al giorno

STACK PERSISTENT

centesimi al mese

AVVISI AWS BUDGETS

10 • 50 • 100 \$

Teardown / l'ordine conta

scripts/aws_teardown.sh

si ferma se kubectl non punta a un cluster EKS

01

kubectl delete namespace

Rimuove i Service e i loro load balancer:
Terraform non li conosce e
bloccherebbero il destroy della VPC.



02

terraform destroy

Distrugge lo stack compute: VPC, EKS,
RDS senza snapshot finale, SQS, ruoli
IRSA e KEDA.



03

Verifica finale

Controlla che nella regione non restino
cluster EKS o istanze RDS accese.

Resta acceso lo stack persistent: modello su S3, immagini su ECR, ruolo della CI e budget. Per ripartire bastano l'apply del compute e il playbook.

Limiti dichiarati / e il passo successivo

01

Niente TLS né autenticazione

API e frontend esposti in HTTP semplice sui load balancer pubblici.

02

Secret Kubernetes ordinari

la stringa di connessione al database non passa da un secret manager.

03

Nessun Cluster Autoscaler

i nodi sono fissi, quindi il tetto dei worker va scelto a mano.

04

Nessuna osservabilità

solo kubectl, log e console AWS: mancano metriche e dashboard.

- Nessuno di questi limiti tocca l'architettura: sono estensioni da aggiungere, non parti da rifare.